

LAPORAN TUGAS BESAR

disusun untuk memenuhi salah satu tugas mata kuliah
CII2J4 Jaringan Komputer

Oleh :

Dafa Rafiansyah	(1301213290)
Gregoreus Marcelino	(1301213483)
Rifqah Amaliyah	(1301213241)



PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
BANDUNG
2022/2023

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Tuhan yang Maha Esa yang telah melimpahkan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan laporan tugas besar pada mata kuliah Jaringan Komputer

Penulis selaku penyusun menyadari bahwa masih banyak kekurangan yang terdapat dalam makalah ini, apabila ada saran dan kritik dari semua pihak sangat penulis perlukan untuk perbaikan ke arah yang lebih baik. Akhirnya penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini.

Bandung, 07 Mei 2023

Penulis

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	2
BAB I	3
1.1. Latar Belakang	3
1.2. Batasan Masalah	3
BAB II	5
2.1. Hasil Program	5
2.2. Analisis Program	5
2.3. Uji Coba Web Server	8
BAB III	11
3.1. Kesimpulan	11
3.2. Kendala	11
Lampiran	12

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam era digital saat ini, kebutuhan akan program jaringan semakin meningkat, terutama program yang terkait dengan internet. Program web server merupakan salah satu jenis program jaringan yang banyak digunakan dalam pengembangan web dan aplikasi berbasis web. TCP socket programming merupakan dasar pembuatan program jaringan yang berjalan di atas protokol TCP/IP. Pemahaman dan kemampuan dalam menggunakan TCP socket programming akan sangat membantu dalam mengembangkan program jaringan, termasuk program web server.

Program web server harus mampu memahami protokol HTTP untuk dapat memberikan respons yang sesuai terhadap permintaan HTTP dari client. Dengan memahami protokol HTTP, kita dapat membuat program web server yang lebih baik dan dapat berfungsi dengan optimal.

Dalam mengembangkan program web server, kita perlu memahami bagaimana web server bekerja dan bagaimana respons HTTP dikirimkan ke client. Dengan demikian, kita dapat membuat program web server yang dapat memberikan respons HTTP yang tepat dan sesuai dengan permintaan dari client.

1.2. Batasan Masalah

Membangun Program Web Server Sederhana Berbasis TCP Socket Programming dengan rincian sebagai berikut :

- a. Implementasi pembuatan TCP socket dan mengaitkannya ke alamat dan port tertentu.

- b. Program web server dapat menerima dan memarsing HTTP request yang dikirimkan oleh browser.
- c. Web server dapat mencari dan mengambil file (dari file system) yang diminta oleh client.
- d. Web server dapat membuat HTTP response message yang terdiri dari header dan konten file yang diminta.
- e. Web server dapat mengirimkan response message yang sudah dibuat ke browser (client) dan dapat ditampilkan dengan benar di sisi client.
- f. Jika file yang diminta oleh client tidak tersedia, web server dapat mengirimkan pesan “404 Not Found” dan dapat ditampilkan dengan benar di sisi client.

BAB II

PEMBAHASAN

2.1. Hasil Program

Dapat diakses pada tautan berikut :

<https://drive.google.com/drive/folders/1CZjy5-YuQ3gvizEKVpJEBt9rVWzbVSo-?usp=sharing>

2.2. Analisis Program

a. TCP Socket

```
serverSocket.bind(('', serverPort))
```

- Code

serverSocket. bind(' ', serverPort)) berfungsi sebagai soket pertama atau pintu pertama yang berguna untuk menunggu klien me request

b. Parsing HTTP Request

```
try:  
    message = connectionSocket.recv(1024).decode()  
    filename = message.split()[1]
```

- Code **connectionSocket.recv(1024)** berguna untuk menerima pesan HTTP dari koneksi socket, yang kemudian pesan tersebut di **decode()** yang mana mengonversi pesan menjadi string.
- Code **split()[1]** berguna untuk memisahkan string, lalu indeks kedua ([1]) dari string tersebut disimpan ke variabel filename.

c. Mengambil file (dari file system) yang diminta oleh client

```
try:
    message = connectionSocket.recv(1024).decode()
    filename = message.split()[1]
    f = open(filename[1:])
    outputdata = f.read()
```

- **open()** adalah sebuah fungsi bawaan Python yang digunakan untuk membuka file dan mengembalikan objek file. Pada code di atas, fungsi **open()** menerima argumen berupa string **filename[1:]** yang merupakan nama file yang akan dibuka. Notasi **[1:]** digunakan untuk mengambil karakter kedua hingga akhir string, karena karakter pertama (indeks 0) dianggap sebagai tanda / atau \ pada path file, tergantung pada sistem operasi yang digunakan.
 - Setelah file berhasil dibuka, fungsi **read()** kemudian digunakan untuk membaca seluruh isi file dan menyimpannya ke dalam variabel **outputdata**. Dengan demikian, pada akhirnya variabel **outputdata** akan berisi string yang merepresentasikan isi file yang telah dibaca.
- d. HTTP response message yang terdiri dari header dan konten file yang diminta

```
#Send one HTTP header line into socket
connectionSocket.send("HTTP/1.1 200 OK\r\n".encode())
connectionSocket.send("Content-Type: text/html\r\n\r\n".encode())

#Send the content of the requested file to the client
for i in range(0, len(outputdata)):
    connectionSocket.send(outputdata[i].encode())
connectionSocket.send("\r\n".encode())
connectionSocket.close()
```

- Code **connectionSocket.send("HTTP/1.1 200 OK\r\n".encode())** berfungsi mengirimkan HTTP response header line ke client. Header line ini menunjukkan bahwa request dari client telah diterima dan diproses oleh server dengan status **"200 OK"**.
- Baris **connectionSocket.send("Content-Type: text/html\r\n\r\n".encode())** akan memberitahu client bahwa tipe konten yang dikirimkan adalah **HTML (Content-Type: text/html)**.
- Baris **for i in range(0, len(outputdata)):**
connectionSocket.send(outputdata[i].encode()) berfungsi mengirimkan isi file yang telah dibaca sebelumnya ke client dalam bentuk byte. Dalam loop **for**, setiap karakter dalam isi file (**outputdata**) diambil satu per satu, diubah menjadi bentuk byte menggunakan fungsi **encode()**, dan dikirimkan ke client menggunakan method **send()** pada **connectionSocket**.
- **connectionSocket.send("\r\n".encode())** akan mengirimkan karakter newline terakhir ke client sebagai penanda akhir dari HTTP response.
- **connectionSocket.close()** kemudian menutup koneksi socket dengan client setelah seluruh isi file telah dikirimkan.

e. Web server mengirimkan response message yang sudah dibuat ke browser client

```
connectSocket.send(bytes('HTTP/1.1 200 OK\r\n\r\n', 'UTF-8'))
for i in range(0, len(outputdata)):
    connectSocket.send(outputdata[i].encode())
connectSocket.send("\r\n".encode())
connectSocket.close()
```


- **ConnectSocket.send** berarti mengirimkan pesan dalam bentuk byte. Pesan yang dikirimkan adalah pesan 200 OK yang berarti file sudah sesuai. Apabila file tidak sesuai atau tidak ditemukan maka pesan yang dikirim adalah 404 File not Found
- **connectSocket.close()**
yang berfungsi sebagai menutup socket koneksi. Oleh karena itu sekarang klien lain dapat menggunakan socket /server

f. Menampilkan pesan “404 Not Found”

```
except IOError:
    #Send response message forfile not found
    connectionSocket.send("HTTP:/1.1 404 Not Found /r/n".encode())
    connectionSocket.send("Content-Type: text/html\r\n\r\n".encode())
    connectionSocket.send("<html><body><h1>404 Not Found</h1></body></html>\r\n".encode())
```

- Kode **connectionSocket.send("HTTP:/1.1 404 Not Found /r/n".encode())** ini berarti mengirimkan respon status line ke klien yang menunjukkan bahwa file yang diminta tidak ditemukan.
- **connectionSocket.send("Content-Type: text/html\r\n\r\n".encode())**, kode ini berarti mengirimkan respon ke klien yang menunjukkan bahwa tipe konten merupakan text/html.
- Kode **connectionSocket.send("<html><body><h1>404 Not Found</h1></body></html>\r\n".encode())** ini berarti mengirimkan pesan body HTTP response ke klien yang berisi bahwa pesan yang diminta tidak ditemukan (404 Not Found).

2.3. Uji Coba Web Server

- Untuk menguji coba Web Server, kita perlu membuat file HTML. Disini kami mengujinya dengan menampilkan kata “HELLO WORLD!”. Berikut kodenya:

```

<> index.html > html > body > h1
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Hello World</title>
5  </head>
6  <body>
7    <h1>HELLO WORLD!</h1>
8  </body>
9  </html>

```

- Kemudian buka terminal lalu jalankan web server dengan menulis python webservertcp.py. Webservertcp.py merupakan file python web server yang telah kami buat.

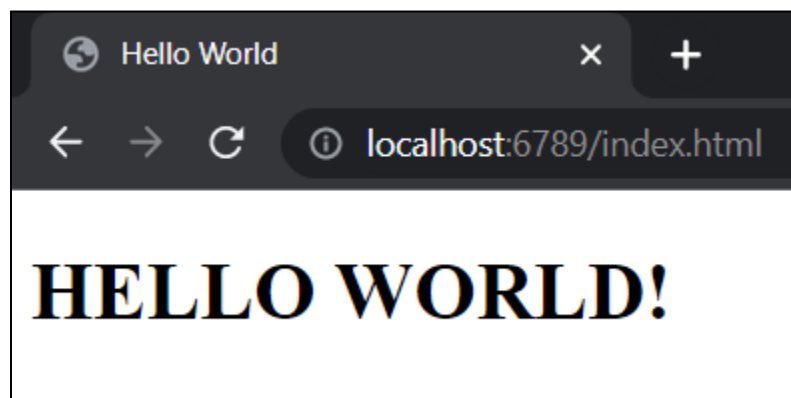
```

Microsoft Windows [Version 10.0.22621.1555]
(c) Microsoft Corporation. All rights reserved.

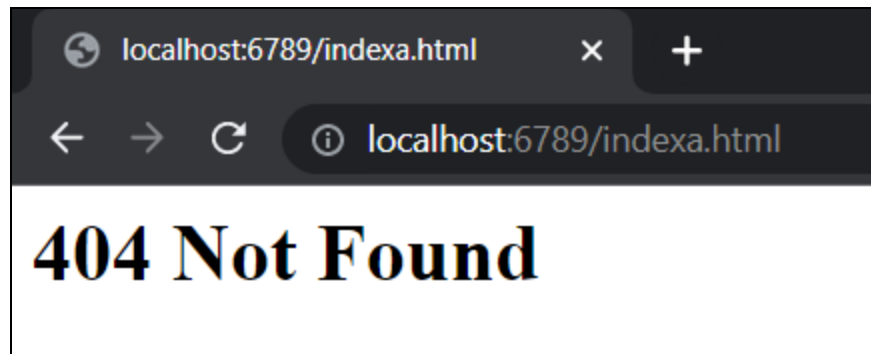
D:\Kuliah\SMT 4\PRAKTIKUM JARKOM\TUBES>python webservertcp.py
Ready to serve...

```

- Buka browser, kemudian tuliskan localhost:6789/index.html
 - 6789 merupakan serverPort yang sudah dibuat di web server
 - index.html merupakan file html yang kita buka.



- Jika file html yang diminta tidak ditemukan, maka server akan memberikan respon "404 Not Found"



BAB III

PENUTUP

3.1. Kesimpulan

Dengan menggunakan TCP Socket Programming, dapat dibangun program web server sederhana yang dapat menerima dan memproses HTTP request dari browser. Program tersebut dapat mengambil file yang diminta oleh client dari file system dan membuat HTTP response message yang terdiri dari header dan konten file yang diminta.

Web server juga dapat mengirimkan response message yang sudah dibuat ke browser (client) dan dapat ditampilkan dengan benar di sisi client. Selain itu, web server juga dapat mengirimkan pesan "404 Not Found" jika file yang diminta oleh client tidak tersedia dan dapat ditampilkan dengan benar di sisi client.

Implementasi praktikum ini memberikan pemahaman tentang cara kerja protokol HTTP dan dasar-dasar pembangunan program web server dengan menggunakan TCP Socket Programming.

3.2. Kendala

-

KONTRIBUSI ANGGOTA

NO	NAMA	TUGAS
1	Dafa Rafiansyah	- Memparsing HTTP request - Menampilkan pesan "404 Not Found
2	Gregoreus Marcelino	- Membuat TCP socket dan mengaitkannya ke alamat dan port tertentu. - Mengirimkan response message yang sudah dibuat ke browser (client) dan dapat ditampilkan dengan benar di sisi client.
3	Rifqah Amaliyah	- Mencari dan mengambil file (dari file system) yang diminta oleh client. - Membuat HTTP response message yang terdiri dari header dan konten file yang diminta.